

Robust Probabilistic Shielding for Safe Offline Reinforcement Learning

Maris F. L. Galesloot
Radboud University
Nijmegen, The Netherlands
maris.galesloot@ru.nl

Thomas Rhemrev
Radboud University
Nijmegen, The Netherlands
thomas.rhemrev@ru.nl

Nils Jansen
Ruhr University & Radboud University
Bochum, Germany
n.jansen@rub.de

ABSTRACT

In *offline* reinforcement learning (RL), we learn policies from fixed datasets without environment interaction. The major challenges are to provide guarantees on the (1) performance and (2) safety of the resulting policy. A technique called *safe policy improvement* (SPI) provides a *performance guarantee*: with high probability, the new policy outperforms a given *baseline policy*, which is assumed to be safe. Orthogonally, in the context of safe RL, a *shield* provides a *safety guarantee* by restricting the action space to those actions that are provably safe with respect to a given *safety-relevant model*. We integrate these paradigms by extending shielding to offline RL, relying solely on the available dataset and knowledge of safe and unsafe states. Then, we shield the policy improvement steps, guaranteeing, with high probability, a safe policy. Experimental results demonstrate that shielded SPI outperforms its unshielded counterpart, improving both average and worst-case performance, particularly in low-data regimes.

KEYWORDS

Offline Reinforcement Learning, Probabilistic Shielding, Safe Reinforcement Learning, Safe Policy Improvement, Robust MDPs

1 INTRODUCTION

In an *online* reinforcement learning (RL) setting, an agent learns to make decisions by interacting with an environment, modeled as a Markov decision process (MDP) [3, 35], to maximize cumulative discounted rewards. In settings where interaction with the environment is infeasible, e.g., when these interactions are costly, dangerous, or otherwise impractical for real-world applications, *offline* RL [27] (also referred to as *batch* RL [15]) provides a feasible alternative. In offline RL, the agent learns a policy solely by analyzing a dataset of transitions in the MDP that was previously collected by a *baseline policy*. To allow for deployment in safety-critical scenarios, a challenge of offline RL is to provide guarantees on the (1) *performance* and (2) *safety* of the resulting policy.

For guaranteeing performance, a promising direction is so-called *safe policy improvement* (SPI) [16, 32, 38]. In offline RL, the SPI problem is to find a policy based on a given dataset that improves over a given (or estimated [39]) baseline policy with high probability [26]. Unfortunately, the number of times each state-action pair must occur in the dataset is prohibitively large to guarantee a tight improvement with high probability. A well-known algorithm that addresses the SPI problem in offline RL is SPI *with baseline bootstrapping* (SPIBB) [26]. In SPIBB, the improvement requirement is relaxed by falling back to the baseline policy for parts of the MDP with low coverage in the dataset. SPIBB rests on the critical assumption that the fallback baseline policy must be safe, since it is

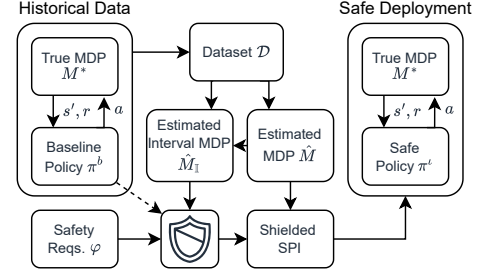


Figure 1: The shielded SPI methodology for safe offline RL.

not always improved. Furthermore, the notion of “safety” in SPIBB merely provides approximate guarantees of improvement over an assumed-to-be-safe policy. No lower bound or guarantees on the actual safety of the resulting policies are guaranteed. Crucially, in many real-world scenarios, there are *safety requirements* to which the policies must adhere, and we must have a *safety guarantee* that these requirements are satisfied.

In *safe* RL, *shielding* [22] is a technique to ensure that an agent does not take actions that would lead to undesirable or unsafe states, providing a *safety guarantee*. For instance, a *reach-avoid* specification governs the probability of reaching safe states while avoiding unsafe states [2]. Then, a *probabilistic shield* restricts the actions that exceed a certain threshold probability of violating such a specification [11, 22]. Shielding is commonly applied in online RL. The major drawback is that the computation of a shield requires the availability of a model in the form of a *safety-relevant* MDP, encompassing all the necessary information for safe behavior [1, 11, 22]. All safety guarantees of shielding rely on this safety-relevant MDP, which is generally unrealistic to assume. To the best of our knowledge, probabilistic shielding has not been studied in an offline RL setting where only a dataset of limited size is available.

Contributions. In this paper, we introduce probabilistic shields to offline RL. Specifically, we integrate probabilistic shielding into SPI. We compute a shield from a given dataset, assuming knowledge of safe and unsafe states. To account for the potential imperfection of the available dataset, we base the shield computation on the construction of a so-called *robust MDP* [45]. These models capture data uncertainty by featuring imprecise transition probabilities via *uncertainty sets* [21, 31]. We construct the robust model as an interval MDP that contains the true data-generating MDP with high probability. Then, we provide, with high probability, a robust bound on the reach-avoid probabilities of the true MDP. Using this

result, we shield the SPI process to ensure it adheres to the safety requirements with high probability while still providing probabilistic guarantees on improvement within the shielded MDP due to SPI. We illustrate our shielding methodology for SPI in Figure 1.

We summarize our contributions as follows:

- (1) **Shields for safe offline RL:** We introduce a novel approach for safe offline RL by computing probabilistic shields on an interval MDP constructed from a given dataset.
- (2) **Shielding from a dataset** (Section 3): We construct an interval MDP from the dataset and define a shield that restricts actions by thresholding worst-case reach-avoid probabilities, providing probabilistic safety guarantees.
- (3) **Integration with SPI(BB)** (Section 4): We incorporate the shield into SPI, focusing specifically on SPIBB, ensuring that the resulting policies adhere to the shield.

Experiments on four benchmarks evaluate the effectiveness of shielding in offline RL, specifically in the context of SPI, in terms of both safety and performance. We show that, in most cases, shielded SPI (1) prevents the computation of unsafe policies and (2) improves overall and worst-case performance over standard SPI.

2 BACKGROUND

First, we introduce basic concepts and notation. A *probability distribution* over a finite set X is a function $\mu: X \rightarrow [0, 1]$ such that $\sum_{x \in X} \mu(x) = 1$. The *support* of a distribution μ is $\text{supp}(\mu) = \{x \in X \mid \mu(x) > 0\}$. Let $\Delta(X)$ be the set of all probability distributions over X , and let $\mathcal{P}(X)$ be the *powerset* of X . For any predicate ϕ , *Iverson brackets*, denoted $[\phi]$, return 1 if ϕ is true and 0 otherwise. For $f: X \rightarrow \Delta(Y)$ and similar, we write $f(y \mid x)$ to denote $f(x)(y)$.

2.1 Markov decision processes

A *Markov decision process* (MDP) models sequential decision-making problems in stochastic environments.

Definition 1. An MDP is a tuple $M = (S, s_0, A, T, R, \gamma)$, where S is the set of *states* with $s_0 \in S$ the *initial state*¹, A is the set of *actions* from which $A(s) \subseteq A$ are *available* at $s \in S$, $T: S \times A \hookrightarrow (S \rightarrow [0, 1])$ is the *partial transition function*, $R: S \times A \rightarrow [-R_{\max}, R_{\max}]$ is the *reward function*, with $R_{\max} \in \mathbb{R}$, and $\gamma \in [0, 1)$ is the *discount factor*.

A stochastic and memoryless *policy* $\pi: S \rightarrow \Delta(A)$ selects the actions to execute. The set of all stochastic and memoryless policies is Π . The *value* of a policy $\pi \in \Pi$ from $s \in S$ is the expected infinite-horizon sum of discounted rewards:

$$V_M^\pi(s) = \mathbb{E}_{M, \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right], \quad (1)$$

where the expectation $\mathbb{E}_{M, \pi}[\cdot]$ is taken over the transition probabilities of the Markov chain induced by following the policy π in the MDP M [35]. Its value follows from the value function $V_M^\pi: S \rightarrow \mathbb{R}$, which is a fixed point of the Bellman equation [3]: $V_M^\pi(s) = \sum_{a \in A} \pi(a \mid s) [R(s, a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V_M^\pi(s')]$. Then, the *performance* $\rho(\pi, M) = V_M^\pi(s_0)$ of a policy π on MDP M is the value of the initial state s_0 . For a set of policies $\Pi' \subseteq \Pi$, a Π' -*optimal policy* $\pi^* \in \Pi'$ maximizes the performance: $\pi^* \in$

¹We present a single initial state, but it generalizes to an initial state distribution.

$\text{argmax}_{\pi \in \Pi'} \rho(\pi, M)$. Analogously to the value V_M^π , the action-value $Q_M^\pi: S \times A \rightarrow \mathbb{R}$ represents the expected value of taking action $a \in A$ in state $s \in S$.

Policy iteration. In this paper, we compute policies using variants of *policy iteration* [35], an algorithm for MDPs consisting of two steps: *policy evaluation* and *policy improvement*. In the evaluation step, we compute the action-value function Q^π . In the improvement step, we use Q^π to update the policy π greedily. These steps are repeated iteratively, updating both $Q^{\pi'} \leftarrow Q^\pi$ and $\pi' \leftarrow \pi$, until $\pi' = \pi$ and convergence to an optimal policy is achieved [35].

Datasets. In RL, we may find an optimal policy by learning the transition and reward function of the MDP. In this paper, we assume, without loss of generality, that the reward function is known. In offline RL, we are restricted to using previously gathered data. A *trajectory* is a finite sequence of states $\tau_H = (s_0, a_0, s_1, a_1, \dots, s_H) \in (S \times A)^* \times S$ where s_0 is the initial state and where $T(s_{t+1} \mid s_t, a_t) > 0$ for $0 \leq t < H$, and $t, H \in \mathbb{N}$. A *dataset* $\mathcal{D}$ consists of a finite number of trajectories. Throughout the paper, we denote by $N_{\mathcal{D}}(s, a, s')$ and $N_{\mathcal{D}}(s, a)$ the number of occurrences of the transition triple (s, a, s') and the state-action pair (s, a) , respectively, in the given dataset of trajectories $\mathcal{D}$.

2.2 Safe policy improvement

The safe policy improvement (SPI) problem [42] in offline RL is to find a policy that improves over a given baseline policy with high probability. Specifically, from a dataset $\mathcal{D}$ and a baseline policy π^b , SPI algorithms aim for a new policy π' that improves upon the baseline policy π^b on the true MDP M^* with high probability:

$$\Pr(\rho(\pi', M^*) \geq \rho(\pi^b, M^*) - \zeta) \geq 1 - \delta_\rho, \quad (2)$$

where $\zeta \in \mathbb{R}$ is the admissible error of the improvement approximation and $\delta_\rho \in [0, 1]$ signifies the required confidence level. Note that ζ is a slack parameter, i.e., improvement is not necessarily achieved. To find a solution to Eq. (2), we define the *maximum likelihood* MDP (MLE-MDP), a tuple $\hat{M} = (S, A, \hat{T}, R, \gamma)$, with a transition function $\hat{T}$ estimated via maximum likelihood as:

$$\hat{T}(s' \mid s, a) = [N_{\mathcal{D}}(s, a) > 0] N_{\mathcal{D}}(s, a, s') / N_{\mathcal{D}}(s, a). \quad (3)$$

Note that in the MLE-MDP, the uncertainty of the point estimate is not taken into account. Given an MLE-MDP and an *error function* $e_{\mathcal{D}}: S \times A \rightarrow \mathbb{R}$, we can define an *admissible set* of MDPs [16] that have a transition function with a bounded ℓ_1 distance $\|\cdot\|_1$ from $\hat{T}$:

$$\Xi_M^{e_{\mathcal{D}}} = \{M \mid \forall s, a: \|T^*(s, a) - \hat{T}(s, a)\|_1 \leq e_{\mathcal{D}}(s, a)\}.$$

The $e_{\mathcal{D}}$ is typically defined from concentration inequalities given the dataset $\mathcal{D}$ such that the true MDP M^* is captured by $\Xi_M^{e_{\mathcal{D}}}$ with probability at least $1 - \delta_\rho$. Then, finding the improved policy π' for Eq. (2) is done through the following formulation [26], where we compute the Π -optimal policy π' on the MLE-MDP $\hat{M}$ such that π' approximately improves over the baseline policy π^b for all MDPs within the admissible set $\Xi_M^{e_{\mathcal{D}}}$:

$$\pi' \in \text{argmax}_{\pi \in \Pi} \rho(\pi, \hat{M}) \quad \text{s.t.} \quad \forall M \in \Xi_M^{e_{\mathcal{D}}}: \rho(\pi, M) \geq \rho(\pi^b, M) - \zeta.$$

Since the admissible set $\Xi_M^{e_{\mathcal{D}}}$ captures the true MDP M^* with probability $1 - \delta_\rho$, it provides a solution to Eq. (2). However, the amount

of data required for each state-action pair to provide this guarantee is substantial [14, 26, 44].

SPIBB. Providing a practical alternative, *safe policy improvement with baseline bootstrapping* (SPIBB) [26] is an algorithm that restricts the policy improvement step to the baseline policy at state-action pairs with high uncertainty [16]. A threshold $N_\lambda \in \mathbb{N}$ is specified to define the number of times a state-action pair must appear in the dataset $\mathcal{D}$ to allow improvement over the baseline. To this end, we have a *bootstrapped set* with *uncertain* state-action pairs that occur less than N_λ times in the dataset $\mathcal{D}$:

$$\mathcal{B} = \{(s, a) \in S \times A \mid N_{\mathcal{D}}(s, a) \leq N_\lambda\}. \quad (4)$$

Typically, N_λ is set as a hyperparameter, together with δ_ρ , and we quantify the improvement approximation ζ [16, 44]. SPIBB optimizes the policy greedily over the expected reward Q_M^π in the MLE-MDP $\hat{M}$, subject to the constraint that $\forall (s, a) \in \mathcal{B}$: $\pi'(a \mid s) = \pi^b(a \mid s)$. The probability mass of non-restricted actions is greedily assigned to the action that maximizes the action-value Q_M^π . Given a state $s \in S$, let $A_{\mathcal{B}}(s) = \{a \in A \mid (s, a) \notin \mathcal{B}\} \subseteq A$ be the set of actions not in the bootstrapped set $\mathcal{B}$. Then, the update is:²

$$\pi'(a \mid s) = \begin{cases} \pi^b(a \mid s) & \text{if } (s, a) \in \mathcal{B}, \\ \sum_{a' \in A_{\mathcal{B}}(s)} \pi^b(a' \mid s) & \text{if } a = \operatorname{argmax}_{a' \in A_{\mathcal{B}}(s)} Q_M^\pi(s, a'), \\ 0 & \text{otherwise.} \end{cases} \quad (5)$$

Then, iterating $\pi \leftarrow \pi'$ from the above Eq. (5) until $\pi' = \pi$, starting from $\pi \leftarrow \pi^b$, then results in the policy π' that satisfies Eq. (2) with a ζ that depends on the chosen N_λ and δ_ρ . In the following, we assume that the baseline policy is known, but this assumption can be lifted by estimating it from the dataset [39], see Section A.

3 PROBABILISTIC SHIELDING IN OFFLINE RL

We introduce *probabilistic shielding* to offline RL, relying solely on a given dataset and knowledge of safe and unsafe states as input.

Type of shield. We focus on *probabilistic shields* [11, 22], and only allow actions for which it is possible to satisfy the safety requirements with a certain probability from the current state onward. Here, a shield is generally a function $A_\square: S \rightarrow \mathcal{P}(A)$ mapping states to sets of allowed actions. That is, we focus on *pre-shielding*, where the shield restricts the set of actions the agent can choose from [24], rather than penalizing actions that lead to unsafe outcomes. In offline RL, we do not have access to the true MDP M^* and thus do not execute the policy before final deployment. Instead, we restrict the actions that can be chosen during policy iteration, ensuring that the shield only allows actions for which it is possible to adhere to a probabilistic safety constraint. In contrast to *permissive policies* [13, 23], we shield purely based on the probabilities of actions violating the safety requirements from the current state, and do not take into account past risks. We refer to Koenighofer et al. [24] for a detailed discussion on the various forms of shielding and their implications.

Overview. The process of constructing the shield from a dataset consists of three steps, described in the following subsections.

- (1) Construct an interval MDP model from the dataset $\mathcal{D}$.
- (2) Specify safety in terms of a reach-avoid specification on a subset of unsafe and target states.
- (3) Compute the worst-case probability of each state-action pair violating the safety specification to construct the shield.

Assumptions. To define a safety specification, we assume we are given a subset of *unsafe* states $S_U \subset S$ to avoid and a subset of *target* states $S_T \subset S$ to reach, with $S_T \cap S_U = \emptyset$. Additionally, we assume knowledge of the underlying graph of the true MDP M^* , i.e., which transitions exist, but not the transition probabilities. While this is not always assumed in SPI methods, it is a common assumption for estimating the IMDP and for analysis of the worst-case [40, 45].

3.1 Constructing an interval MDP

Here, we construct an *interval* MDP (IMDP) with an interval-based *uncertainty set* that captures the true MDP with high probability, similar to the admissible set for SPI.

Definition 2 (IMDP). An IMDP is a tuple $M_I = (S, s_0, A, T_I, R, \gamma)$. Here S, s_0, A, R , and γ are the same as in an MDP. The partial *interval transition function* is defined as $T_I: S \times A \hookrightarrow (S \rightarrow \mathbb{I} \cup \{0\})$, where $\mathbb{I}$ is the set of *probability intervals* $\mathbb{I} = \{[l, u] \mid 0 < l < u \leq 1\}$.

The interval transition function T_I assigns either a probability interval or an exact probability of 0 to each transition if the transition does not exist. In contrast to the ball-shaped admissible set formalized using the ℓ_1 norm, we construct a box-shaped uncertainty set, which allows an IMDP. Note that in SPIBB, the admissible set is not constructed but solely defined to formalize a statistical guarantee. Conversely, we explicitly construct the IMDP to compute a shield that provides robust safety guarantees based on the worst-case probabilities. We construct IMDPs because they enable efficient analysis of worst-case probabilities, providing a lower bound on optimal probabilities, which we use to design the shield.

We construct interval ranges such that the true MDP M^* is captured with probability at least $1 - \delta_I$. We use the mode of a Dirichlet distribution as a point estimate $\hat{T}$ [40]. We provide details in Section B. We distribute δ_I over transitions via the union bound: $\delta_T = \delta_I / \sum_{s,a} |\operatorname{supp}(T(s,a))|$, where $|\operatorname{supp}(T(s,a))|$ is the number of successor states. Given a point estimate $\hat{T}$, and a confidence δ_T , we find the interval size η_T such that the true transition probability of T^* falls within the interval with a probability at least $1 - \delta_T$. We find it each for transition (s, a, s'_i) , using Hoeffding's inequality [20]: $\eta_T = \sqrt{\log[2/\delta_T] / 2N_{\mathcal{D}}(s,a)}$. Intuitively, since we know the sample sizes and have a desired confidence level δ_T , we determine the margin η_T around the point estimate. Using this η_T , we construct the interval for the state-action-state triplet (s, a, s'_i) to be $\hat{T}_I(s'_i \mid s, a) = [l, u]$, where $l = \max(\xi, \hat{T}(s'_i \mid s, a) - \eta_T)$ and $u = \min(\hat{T}(s'_i \mid s, a) + \eta_T, 1)$. Here, $\xi \in (0, 1)$ is to ensure that the interval bounds u and l are in $[\xi, 1]$. Repeating this process for every transition in the graph yields the IMDP M_I conditioned on the dataset $\mathcal{D}$. We achieve a guarantee on the level of the whole model by applying the *union bound* across the transitions of the IMDP.

²In cases with no unique maximizing action a' , tie-breaking in the argmax is arbitrary.

Corollary 1. The transition probabilities T^* of the true MDP M^* are within the transition intervals $\hat{T}_I$ of the interval MDP $\hat{M}_I$ with probability at least $1 - \delta_I$. That is, we have $\Pr(M^* \in \hat{M}_I) \geq 1 - \delta_I$.

Note that tighter intervals may be possible using other statistical methods [28, 44]. Still, we use Hoeffding’s inequality here for its simplicity. To ensure robustness, in the subsequent steps, we take a pessimistic view of the interval MDP, accounting for the worst-case realization of the transition probabilities within the intervals to compute the shield, providing a lower bound with high probability.

3.2 Defining the probabilistic specification

The second step in the process of constructing a shield is defining a probabilistic safety specification. We encode our specification with a *temporal logic formula* $\varphi = \neg S_U \cup S_T$ (read: “not unsafe until target”) [2]. Specifically, we formulate a *reach-avoid* formula, but an “avoid” formula may also be used. Then, we formulate a probabilistic reach-avoid specification on the formula φ .

Given the estimated IMDP $\hat{M}_I$ and the formula φ , we define our specification based on the maximal reach-avoid probabilities. Since the IMDP effectively defines ranges of probabilities, we robustly address this uncertainty by considering the worst case [34]:

$$\mathbb{P}_{\maxmin}^{\hat{M}_I, \Pi, \varphi}(s) = \max_{\pi \in \Pi} \min_{M \in \hat{M}_I} \mathbb{P}_M^{\pi, \varphi}(s),$$

where $\mathbb{P}_M^{\pi, \varphi}(s)$ is the probability of satisfying the formula φ under policy π in the MDP M , starting from the state $s \in S$ [2]. Intuitively, $\mathbb{P}_{\maxmin}^{\hat{M}_I, \Pi, \varphi}(s_0)$ represents, from the initial state s_0 , the probability that an agent will not enter any unsafe state before reaching a target state, when the agent selects actions that are optimal in terms of reach-avoid in the worst-case. In the probabilistic *specification*, the probability must satisfy a *safety threshold* $\theta \in [0, 1]$ from the initial state s_0 . We disregard the past risk for tractability purposes [24], departing from semantics where the shield enforces the specification in the initial state of the MDP [11, 23]. Instead, the shield ensures, with high probability, that for each state $s \in S$, the optimal worst-case reach-avoid probability adheres to the threshold. Specifically, the shield restricts actions such that, from that point onward and when ignoring past risks, the specification can still be satisfied.

3.3 Computing and constructing the shield

In the final step, we compute, for all states, the worst-case probabilities of violating the specification on the IMDP. The worst-case probabilities can be induced by either the lower or upper bounds of the individual transition probabilities, hence the need for intervals. We design the shield by thresholding the worst-case probabilities according to the specification.

Computing the probabilities. Given the specification and the IMDP $\hat{M}_I$, we find the associated function that map states $\mathbb{V}_{\hat{M}_I}: S \rightarrow [0, 1]$ and state-action pairs $\mathbb{Q}_{\hat{M}_I}: S \times A \rightarrow [0, 1]$ to maximal worst-case probabilities. We set $\mathbb{V}_{\hat{M}_I}(s) = 1$ if $s \in S_T$ and $\mathbb{V}_{\hat{M}_I}(s) = 0$ if $s \in S_U$. Then, we have for $s \in S \setminus (S_T \cup S_U)$, the equations:

$$\begin{aligned} \mathbb{V}_{\hat{M}_I}(s) &= \max_{a \in A} \mathbb{Q}_{\hat{M}_I}(s, a), \\ \mathbb{Q}_{\hat{M}_I}(s, a) &= \inf_{\underline{T}(s, a) \in \hat{T}_I(s, a) \cap \Delta(S)} \sum_{s' \in S} \underline{T}(s' | s, a) \mathbb{V}_{\hat{M}_I}(s'). \end{aligned} \quad (6)$$

The fixed points $\mathbb{V}_{\hat{M}_I}^*$ and $\mathbb{Q}_{\hat{M}_I}^*$, represent the *robust optimal*, i.e., optimal worst-case reach-avoid probabilities, such that $\mathbb{V}_{\hat{M}_I}^*(s) = \mathbb{P}_{\maxmin}^{\hat{M}_I, \Pi, \varphi}(s)$. Specifically, the action-probability $\mathbb{Q}_{\hat{M}_I}^*(s, a)$ represents, in expectation over successor states, the worst-case reach-avoid probability at any state $s \in S$ when taking action $a \in A$ and following the robust optimal reach-avoid actions afterwards [34].

For a given MDP M , the optimal (non-robust) reach-avoid probabilities $\mathbb{V}_M: S \rightarrow [0, 1]$ with $\mathbb{V}_M(s) = \max_{\pi \in \Pi} \mathbb{P}_M^{\pi, \varphi}(s)$ and $\mathbb{Q}_M: S \times A \rightarrow [0, 1]$ with $\mathbb{Q}_M(s, a) = \sum_{s'} T(s' | s, a) \mathbb{V}_M(s')$ for $s \notin S_U \cup S_T$ follow the computation of $\mathbb{V}_{\hat{M}_I}^*, \mathbb{Q}_{\hat{M}_I}^*$ by removing the inner infimum in Eq. (6) as T is known. We provide the following lower bound:

Lemma 1. Robust optimal reach-avoid probabilities $\mathbb{V}_{\hat{M}_I}^*$ provide a lower bound on the true reach-avoid probabilities $\mathbb{V}_{M^*}^*$ with probability at least $1 - \delta_I$: $\forall s \in S: \Pr(\mathbb{V}_{\hat{M}_I}^*(s) \leq \mathbb{V}_{M^*}^*(s)) \geq 1 - \delta_I$.

Corollary 2. For $s \in S$ and $a \in A(s)$: $\mathbb{Q}_{\hat{M}_I}^*(s, a) \leq \mathbb{Q}_{M^*}(s, a)$.

Corollary 2 is due to Lemma 1 and Corollary 1. For IMDPs, the inner infimum of Equation (6) can be solved efficiently through a bisection method in combination with robust dynamic programming [21, 31, 34, 46]. Notably, computing these worst-case maximal probabilities for IMDPs is supported out of the box by probabilistic model checkers, e.g., PRISM [25] and STORM [12].

Designing the shield. The *safety threshold* $\theta \in [0, 1]$ is a settable parameter to bound the robust reach-avoid probabilities. Any state-action pair for which the probability is above this safety threshold is considered unsafe. Thus, we shield with respect to the robust optimal reach-avoid probabilities in the IMDP. That is, with high probability, the shield allows any action for which it is possible to satisfy the threshold when ignoring past risk [24]. We define the notion of a safe action below.

Definition 3 (θ -Safe action). An action $a \in A$ for a state $s \in S$ is a θ -safe action in state s if from state s , the probability of violating the reach-avoid specification in the IMDP is at most θ , i.e., it satisfies $\mathbb{Q}_{\hat{M}_I}^*(s, a) > 1 - \theta$. Otherwise, action a is θ -unsafe in state s .

Assumption 1 (Feasibility). We assume the threshold $\theta \in [0, 1]$ is *feasible*, i.e., a θ -safe action is always available. Otherwise, it is infeasible to satisfy the safety specification, and no shield exists. That is, $\forall s \in S, \exists a \in A: \mathbb{Q}_{\hat{M}_I}^*(s, a) > 1 - \theta$.

Next, we define the shield resulting from Definition 3.

Definition 4 (θ -Shield). A θ -shield is a function $A_{\square}: S \rightarrow \mathcal{P}(A)$ that returns the set of θ -safe actions given a state $s \in S$, defined as:

$$A_{\square}(s) = \{a \in A \mid \mathbb{Q}_{\hat{M}_I}^*(s, a) > 1 - \theta\} \subseteq A.$$

Conversely, let $A_{\square}(s) = A \setminus A_{\square}(s)$ be the set of θ -unsafe actions such that $\forall s: A_{\square}(s) \cup A_{\square}(s) = A$ and $A_{\square}(s) \cap A_{\square}(s) = \emptyset$.

Consequently, we define a policy to be safe if, for all states, its support is a subset of the actions allowed by the shield.

Definition 5 (θ -Safe policy). A policy $\pi \in \Pi$ is θ -safe if its support contains only θ -safe actions: $\forall s \in S: \text{supp}(\pi(s)) \subseteq A_{\square}(s)$.

Remark 1 (Infeasibility). In practice, when there exist states where the threshold is infeasible, we select the action closest to the threshold, and we allow all actions with probabilities within a range $\kappa \in [0, 1]$ around the probability of the closest action.

3.4 Safety guarantees

Finally, we study the theoretical properties of the shield. In the above, we constructed the IMDP to compute the shield. Now, we argue about guarantees of the shield on the true MDP M^* .

Definition 6. The *shielded-MDP* $M_{\sqcup} = \langle S, A, T_{\sqcup}, R, \gamma \rangle$ for an MDP M and shield $A_{\sqcup}$ is an MDP where $T_{\sqcup}(s, a) = T(s, a)$ if $a \in A_{\sqcup}(s)$ and $T_{\sqcup}(s, a) = \perp$ otherwise, i.e., a does not exist in the latter case.

Next, we present a probabilistic guarantee that the shield contains only safe actions with high probability.

Theorem 1. Given a confidence level $\delta_{\bar{I}}$, and feasible threshold θ , and conditioned on a dataset $\mathcal{D}$, a θ -shield $A_{\sqcup}$ solely maps to θ -safe actions on the true MDP M^* with probability at least $1 - \delta_{\bar{I}}$. Formally, starting from any state $s \in S$, the probability that the shielded action $a \in A_{\sqcup}(s)$ is also θ -safe on the true MDP M^* is:

$$\Pr \left(\mathbb{Q}_{M^*}^*(s, a) \geq 1 - \theta \right) \geq 1 - \delta_{\bar{I}}.$$

Proof 1. We defined an action $a \in A_{\sqcup}(s)$ to be a θ -safe action from state $s \in S$ if $\mathbb{Q}_{M_{\bar{I}}}^*(s, a) \geq 1 - \theta$. From Corollary 2, we derive that with probability at least $1 - \delta_{\bar{I}}$, we have that $\mathbb{Q}_{M_{\bar{I}}}^*(s, a) \leq \mathbb{Q}_{M^*}^*(s, a)$. $\square$

The proof follows from Corollary 2 and our definition of θ -safety with respect to robust reach-avoid probabilities. Namely, condition on any $\mathcal{D}$, the IMDP $\hat{M}_{\bar{I}}$ contains the true MDP M^* with probability at least $1 - \delta_{\bar{I}}$. Thus, the shield on the IMDP also provides, with high probability, worst-case safety guarantees on the true MDP M^* .

Corollary 3. Safety with respect to the reach-avoid probabilities on the true MDP M^* is lower bounded by the shielded MDP $M_{\sqcup}^*$: $\forall s \in S: \mathbb{V}_{M_{\sqcup}^*}^*(s) \leq \mathbb{V}_{M^*}^*(s)$ with probability at least $1 - \delta_{\bar{I}}$.

Equality is not guaranteed in the above, so the shield may be more conservative than a shield using the true probabilities. Nonetheless, Theorem 1 can be lifted to affect shielded policies; i.e., if any action deemed unsafe from the shield is ruled out, then the same probabilistic guarantee holds for the resulting policy.

4 PROBABILISTIC SHIELDING FOR SPI

In this section, we integrate shields into SPIBB, yielding a new algorithm variant that we call *shielded SPIBB* (SPIBB $_{\sqcup}$). First, we describe how we ensure the safety of the policies resulting from SPIBB $_{\sqcup}$ by restricting the policy space and shielding the baseline. Then, we analyze the policies that SPIBB $_{\sqcup}$ produces.

Other SPI algorithms. While we focus the presentation on SPIBB, the shield may also be similarly applied to other SPI algorithms. For instance, DUIPI [37] is an SPI algorithm that penalizes the action-values of the MLE-MDP with high uncertainty by computing their variances. We also construct a shielded version of DUIPI, which we name *shielded DUIPI* (DUIPI $_{\sqcup}$), which we describe in Section D.1.

4.1 Applying the shield to SPIBB

We enforce the shield on the baseline policy and incorporate it into the SPIBB algorithm. Applying the shield ensures that resulting policies have no support over unsafe actions.

Shielding the baseline. Firstly, we enforce the shield on the baseline policy π^b , creating a *shielded baseline* $\pi_{\sqcup}^b \in \Pi$.

Definition 7 (Shielded baseline). Given the baseline policy $\pi^b \in \Pi$ and the shield $A_{\sqcup}: S \rightarrow \mathcal{P}(A)$, the *shielded baseline* $\pi_{\sqcup}^b \in \Pi$ with probabilities for $a \in A$ given $s \in S$, with $A_{\sqcup}^s = A_{\sqcup}(s)$ is defined as:

$$\pi_{\sqcup}^b(a | s) = [a \in A_{\sqcup}(s)] \left(\pi^b(a | s) + \sum_{a' \in A_{\sqcup}^s} \pi^b(a' | s) \cdot (|A_{\sqcup}(s)|)^{-1} \right).$$

For each state $s \in S$, any θ -unsafe action $a \notin A_{\sqcup}(s)$ has a probability of zero, and the probability mass of unsafe actions is distributed equally across the remaining safe actions. That is, for safe actions $a \in A_{\sqcup}(s)$, we add the sum of probability mass $\pi^b(a' | s)$ of all unsafe actions $a' \in A_{\sqcup}^s$ in the baseline policy, divided by the number of safe actions $|A_{\sqcup}(s)|$.

Corollary 4. The shielded baseline $\pi_{\sqcup}^b$ is θ -safe.

Shielding SPIBB. First, we amend the bootstrapped set $\mathcal{B}$. In addition to the state-action pairs that appear fewer than $N_{\wedge}$ times in the dataset $\mathcal{D}$, we include state-action pairs deemed unsafe by the shield, regardless of their frequency of occurrence in the dataset.

$$\mathcal{B}_{\sqcup} = \{(s, a) \in S \times A : N_{\mathcal{D}}(s, a) \leq N_{\wedge} \vee a \notin A_{\sqcup}(s)\}$$

Consequently, for these state-action pairs, the shielded baseline is used, which, by Corollary 4, adheres to the shield.

Recall Eq. (5). Similarly, in SPIBB $_{\sqcup}$ we optimize the policy subject to: $\forall (s, a) \in \mathcal{B}_{\sqcup}: \pi'(a | s) = \pi_{\sqcup}^b(a | s)$. Then, SPIBB $_{\sqcup}$ addresses a SPI problem akin to Equation (2) where the objective is to find a policy $\pi'_{\sqcup}$ that approximately improves over the shielded baseline $\pi_{\sqcup}^b$ on the shielded MLE-MDP $\hat{M}_{\sqcup}$ with high probability. Thus, the policy improvement step of SPIBB $_{\sqcup}$ that adheres to the shield is:

$$\pi'(a | s) = \begin{cases} \pi_{\sqcup}^b(a | s) & \text{if } (s, a) \in \mathcal{B}_{\sqcup}, \\ \sum_{a' \in A_{\mathcal{B}_{\sqcup}}(s)} \pi_{\sqcup}^b(a' | s) & \text{if } a = \underset{a' \in A_{\mathcal{B}_{\sqcup}}(s)}{\operatorname{argmax}} Q_{\hat{M}_{\sqcup}}^{\pi}(s, a'), \\ 0 & \text{otherwise.} \end{cases} \quad (7)$$

Intuitively, SPIBB $_{\sqcup}$ greedily maximizes the action-value $Q_{\hat{M}_{\sqcup}}^{\pi}$ across the safe actions of the shielded MLE-MDP outside of the set $\mathcal{B}_{\sqcup}$.

4.2 Analysis of shielded SPIBB

Let $\Pi_{\mathcal{B}_{\sqcup}} = \{\pi: S \rightarrow \Delta(A) \mid \forall s, a \in \mathcal{B}_{\sqcup}: \pi(a | s) = \pi_{\sqcup}^b(a | s)\}$ be the set of policies under the bootstrapped set $\mathcal{B}_{\sqcup}$. By definition and Corollary 4, any policy $\pi \in \Pi_{\mathcal{B}_{\sqcup}}$ is θ -safe. First, we formalize what policy iteration with Eq. (7) amounts to. In SPIBB $_{\sqcup}$, policy iteration $\pi' \leftarrow \pi$ with Eq. (7), starting from $\pi_{\sqcup}^b$ converges to $\pi'_{\sqcup}$ when $\pi' = \pi$, formalized below:

Lemma 2. SPIBB $_{\square}$ finds a $\Pi_{\mathcal{B}_{\square}}$ -optimal policy for the shielded MLE-MDP $\hat{M}_{\square}$, i.e., the policy: $\pi_{\square}^l \in \operatorname{argmax}_{\pi \in \Pi_{\mathcal{B}_{\square}}} \rho(\pi, \hat{M}_{\square})$.

The above is a simple application of policy iteration to the set of shielded policies on the MLE-MDP [26, 35]. Next, we formalize that SPIBB $_{\square}$ produces policies that adhere to the shield, i.e., are θ -safe.

Lemma 3 (SPIBB $_{\square}$ is θ -safe). SPIBB $_{\square}$ policies $\pi_{\square}^l \in \Pi$ are θ -safe in the true MDP M^* with probability at least $1 - \delta_i$.

Lemma 3 is with high probability, since the shield is based on the worst-case that accounts for the true MDP M^* with a probability of at least $1 - \delta_i$, following Theorem 1. In summary, SPIBB $_{\square}$ provides a practical algorithm that produces policies that adhere to the shield constructed from the dataset in Section 3. SPIBB $_{\square}$ shields the baseline policy and, by adding unsafe state-action pairs to the bootstrapped set, greedily optimizes the policy over shielded actions with respect to the action-values of the shielded MLE-MDP $\hat{M}_{\square}$.

5 EXPERIMENTAL EVALUATION

In this section, we evaluate the shielded SPI methods against their non-shielded counterparts using four benchmarks. We aim to answer the following three questions in the context of SPI methods:

- (Q1) How does shielding affect the sample complexity and performance of the resulting policies over the size of the dataset?
- (Q2) How does shielding affect worst-case performance, in terms of the average of the worst runs (measured in 1%-CVaR)?
- (Q3) How does shielding affect the safety of the resulting policies?

5.1 Experimental setup

Methods. In addition to SPIBB and SPIBB $_{\square}$ from Section 4, we consider DUIPI [37]. DUIPI is an SPI method that addresses uncertainty by computing the variance of the estimate of the transition function and propagating it through the action-values. DUIPI penalizes the value of actions with uncertain outcomes by subtracting variance. In a similar fashion to SPIBB $_{\square}$, we create a shielded version DUIPI $_{\square}$ by restricting the actions that can be chosen and by initializing with the shielded baseline $\pi_{\square}^b$. We provide more details on DUIPI and DUIPI $_{\square}$ in Section D.1.

Infrastructure. All experiments were conducted on the CPU of a Mac Studio with an M1 Ultra (128 GB unified memory). We build on the implementation of SPIBB and DUIPI by Scholl et al. [38] and use their experimental framework in Python as the basis for ours. The code is available at: anonymous.4open.science/r/shielded-spi. We use the probabilistic model checker STORM [12] to build and analyze the IMDP used to construct the shield.

Baseline policies. The baseline policies $\pi^b \in \Pi$ are generated in a mixture with $\epsilon \in [0, 1]$ such that $\pi^b = \epsilon\pi^h + (1 - \epsilon)\tilde{\pi}$, where $\pi^h \in \Pi$ is a *heuristic policy* and $\tilde{\pi} \in \Pi$ is the *uniform random policy* with $\forall(s, a) \in S \times A: \tilde{\pi}(a | s) = 1/|A|$.

Experiments. Performing one experiment consists of the following steps, resembling steps in Figure 1. First, we gather a dataset $\mathcal{D}$ by executing the baseline policy π^b on the true MDP M^* . Second, we estimate the baseline policy π^b from the data [39] as described in Section A. Third, we run the methods described above, using the

Table 1: Dimensions and hyperparameters of the benchmarks. We set $\gamma = 0.95$ and $\delta_i = 0.1$ across.

Benchmarks	$ S $	$ A $	$N_{\wedge}$	θ	κ	ϵ
<i>Random MDPs</i>	50	4	3	0.2	0.05	0.5
<i>Wet Chicken</i>	26	5	7	0.2	0.05	0.05
<i>Frozen Lake</i>	64	4	3	0.2	0.02	0.5
<i>Pacman</i>	118k	4	3	0.01	0.01	0.5

data and the (shielded) baseline policy to compute a new policy π . Finally, we evaluate the performance $\rho(\pi, M^*)$ of the resulting policy π on the true MDP M^* . Each experiment is performed 1000 times, and we report both the average performance and the 1%-CVaR performance, which is the average of the worst 10 runs. Important hyperparameters are listed in Table 1.

Benchmarks. The *Random MDPs* benchmark [30] consists of randomly generated MDPs with $|S| = 100$ states, where each state has four actions that lead to four different states with a random probability. The *Wet Chicken* benchmark [18] models an agent controlling a boat on a river adjacent to a waterfall, where the objective is to stay as close to the edge of the waterfall without falling off. The *Frozen Lake* benchmark [43] involves navigating an 8×8 grid to reach a goal state while avoiding hazardous hole states. The above benchmarks used in SPI are small; therefore, we construct the larger *Pacman* benchmark, inspired by Alshiekh et al. [1]; a 7×7 maze where the agent, able to move in the four cardinal directions, starts in a corner and aims to reach a goal state while avoiding two randomly moving ghosts. We provide dimensions in Table 1 and detailed descriptions in Section D of the supplemental material.

5.2 Results and analysis

Section 5.1 shows the average and 1%-CVaR performance of the shielded and non-shielded DUIPI and SPIBB methods plotted against the size of the dataset $\mathcal{D}$, using a log-scale for the x -axis depicting dataset size. Note that for the episodic Pacman, Frozen Lake, and Random MDPs benchmarks, the dataset size refers to the number of trajectories in $\mathcal{D}$. In contrast, for the non-episodic Wet Chicken benchmark, it refers to the number of transitions in the dataset. Additionally, we plot the performance of the optimal, baseline, and shielded baseline policies. Below, we analyze the results as answers to the three research questions.

(Q1): *Data complexity.* Both in terms of average and worst-case performance, **the shielded methods consistently outperform their non-shielded counterparts in low-data regimes**, particularly for smaller datasets. While both the shielded and non-shielded methods tend to converge similarly with sufficiently large datasets, the shielded versions achieve this level of performance with smaller datasets and generally converge faster.

The effectiveness of the shield on small datasets can be attributed to the fact that learning algorithms often struggle when data is scarce, as they lack sufficient information to accurately estimate the value of actions. This lack of data increases the likelihood that the agent will select unsafe or suboptimal actions. The shield mitigates this risk by ensuring that the optimization is applied only to safe

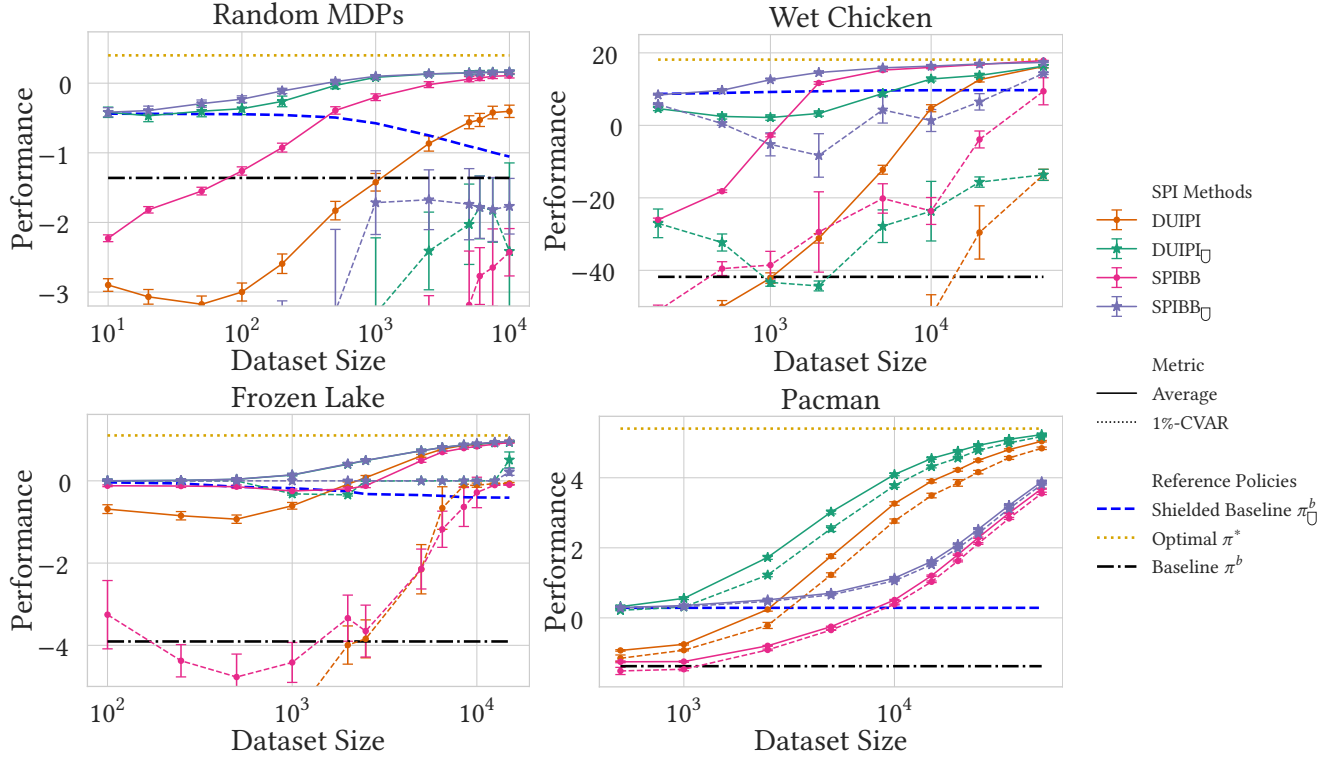


Figure 2: Performance $\rho(\pi, M^*)$ on the true MDP M^* of the policies $\pi \in \Pi$ produced by the shielded ($\star$) and non-shielded ($\bullet$) SPI methods plotted against the number of trajectories in the dataset $\mathcal{D}$. We report average (solid) and 1%-CVaR (dotted) across runs. The x-axis is on a log scale, and error bars denote 95% confidence intervals. The y-axis is clipped for visibility purposes; full ranges are in Section C. The (shielded) baseline and optimal performances on the true MDP M^* are included for reference.

actions, resulting in fewer unsafe outcomes, as discussed in the answer to Q3, which in turn leads to a higher overall reward. Moreover, SPI methods fall back to the baseline policy for state-action pairs with limited data. By shielding the baseline policy, the fall-back will always select safe actions, providing an additional layer of protection and resulting in better overall performance with less data. In summary, **shielded SPI outperforms standard SPI with low-data regimes, generally converging faster.**

(Q2): Worst-case performance. The 1%-CVaR results show that the shielded methods generally outperform their non-shielded counterparts in terms of worst-case performance (see also Figure 4 in Section C for the full range 1%-CVaR results). Generally, we observe that the gains from the shield are larger in terms of 1%-CVaR, indicating that the shield makes the algorithms more robust in the worst 1% of runs, potentially when the datasets are low-quality.

The effect of the shield on the 1%-CVaR is especially pronounced in the Pacman benchmark, which is also the largest. Here, the 1%-CVaR performance of the shielded methods significantly outperforms their non-shielded counterparts. Furthermore, we emphasize that the 1%-CVaR performance of the shielded methods is superior to the average performance of the non-shielded methods across all dataset sizes. Thus, the worst runs of the method with the shield are still better than the average of the runs without the shield. For the other three benchmarks, we observe a general improvement in

the shielded methods when comparing 1%-CVaR. In summary, we conclude that **our shielding approach significantly improves worst-case outcomes as evidenced by 1%-CVaR performance.**

(Q3): Safety. In the four benchmarks used in our experiments, all unsafe states are associated with a negative reward, while all safe states have a reward of zero or greater. This means that if a policy’s performance is negative, it has a positive probability of reaching an unsafe state.

In three of the four benchmarks, we observe that the average performance of shielded SPI methods remains above zero. In the random MDPs benchmark, the shielded methods, as well as the shielded baseline, start with a performance slightly below zero. Here, many actions inherently carry a non-negligible probability of eventually leading to a trap, making it challenging, if not impossible, to identify a policy that completely guarantees safety. Still, we observe that the shielded SPI methods achieve a better performance.

The Frozen Lake benchmark demonstrates that the shield helps the SPI methods avoid unsafe actions. Here, shielded methods almost never produce policies that yield negative rewards, indicating that they consistently avoid unsafe behaviors, regardless of dataset size. With small datasets, the shielded methods converge to conservative but safe policies that yield a reward of just above zero. While this policy does not reach the goal state, and thus does not earn a positive reward, it also avoids any unsafe states. In contrast,

the non-shielded methods learn policies that not only fail to reach the goal state but also contain unsafe actions. Even with larger datasets, shielded methods retain this advantage, achieving higher rewards because they carry a lower risk of unsafe outcomes than their non-shielded counterparts. Achieving the same performance as the non-shielded methods with larger datasets demonstrates that, in our benchmarks, the shield does not hinder the computation of near-optimal policies. In summary, **the shielded methods achieve safer policies in low-data regimes while remaining competitive with larger datasets.**

5.3 Discussion

The results demonstrate that probabilistic shielding can effectively enhance offline SPI methods. By constraining learned policies to avoid actions that violate reach-avoid specifications with high probability, our approach yields better average and worst-case performance, particularly in settings where data is scarce. These findings provide positive answers to (Q1) and (Q2). Additionally, the results also show that the shielded SPI methods learn safer policies with fewer unsafe outcomes, positively answering (Q3).

The performance of the shield depends on the quality of the estimated IMDP $\hat{M}_I$ learned from data. In cases where the dataset is too sparse or the interval estimates are poorly calibrated, the transition intervals might be too wide or inaccurate to be useful in shielding. Since the shield utilizes probabilities computed from the worst-case transition function within the IMDP, wide intervals may result in an overly conservative shield. On the other hand, intervals that are too narrow may not capture the actual transition dynamics and therefore fail to identify genuine risks.

In some benchmarks, the shielded baseline performance declines for larger datasets. This happens because we use a fixed safety threshold θ for all dataset sizes. As the dataset size increases, the estimated intervals for each transition typically get smaller. Because we use the worst-case transitions within the uncertainty set, a larger dataset generally results in a lower probability of violating the reach-avoid specification. Thus, more actions may be found to be θ -safe, and the shielded baseline incurs lower performance. It is challenging to find a threshold value that works for both small datasets with more conservative probabilities and larger datasets with more accurate probabilities. Since the shield is most effective for smaller datasets, we selected a threshold value that works well for these dataset sizes, and kept it consistent across all dataset sizes.

Finally, we note that a limitation of the shield’s design is that there may be a trade-off between safety and optimality. By construction, the shield restricts the action space based on worst-case reach-avoid probabilities. By constraining action choices based on worst-case estimates, the shield may block actions that offer high rewards but also entail a small risk of failure. This conservative behavior can lead to suboptimal policies, particularly when the safety threshold and confidence parameters are poorly calibrated.

6 RELATED WORK

Here, we provide an overview of related literature in the fields of (probabilistic) shielding (in online RL) and SPI (in offline RL).

Shielding. Alshiekh et al. [1] synthesize shields by constructing a *safety game* that combines a *linear temporal logic* specification with

an abstract model of the environment. Solving the game prevents unsafe actions while allowing the agent to learn optimally. Jansen et al. [22] calculate probabilistic shields, shielding actions that are relative to the optimal safety. Carr et al. [8] calculate reach-avoid shields, yet in the context of deep RL and under partial observability. Pranger et al. [33] synthesize a shield adaptively, using separate cost functions for performance and interference, and calculate a shield based on a weighted cost objective. Court et al. [11] guarantee probabilistic safety by constructing a continuous MDP that embeds the probabilistic safety constraints and apply deep reinforcement learning to find a safe policy. Notable extensions of shielding include hybrid [5, 6] and multi-agent systems [4, 7, 29]. All the above assume some form of a *safety-relevant* model, where the part of the MDP relevant to safety is assumed to be fully known. In contrast, Tappler et al. [41] compute shields for exploration of an unknown MDP using automata learning. Furthermore, in model-free (deep) RL, He et al. [19] introduce *latent shielding*, simulating future trajectories in a latent space and estimating the likelihood of an action leading to a safety violation. Goodall and Belardinelli [17] build on this idea and introduce *approximate model-based shielding*, to look-ahead for safety. Yet, all the above literature focuses on shielding in online scenarios, where the shields limit the states the agent can visit during exploration. Instead, we apply shields to constrain the space of policies that can be learned in offline RL.

Safe Policy Improvement. In SPI [32] for offline RL, prior work has focused on improving an (assumed to be safe) baseline policy with high confidence. HCPI [42] uses a high-confidence lower bound on the expected return of a candidate policy using importance sampling estimates. Ghavamzadeh et al. [16] formalize and provide approximations to the robust policy improvement problem, also introducing a method that penalizes the reward function based on how often a state-action pair occurs in the dataset. Chandak et al. [10] study SPI in non-stationary MDPs. In the context of SPIBB, Simão et al. [39] estimate the baseline policy from data, and Satija et al. [36] study multiple objectives, incorporating constraints on expected costs. Furthermore, Wienhöft et al. [44] address the sample complexity of SPI, Engelen et al. [14] exploit parametric and graphical structure and Castellini et al. [9] scale up using Monte Carlo tree search. Nadjahi et al. [30] introduce Soft-SPIBB, relaxing constraints on policy search, and Scholl et al. [38] identify and propose solutions for limitations of Soft-SPIBB.

7 CONCLUSION

We presented a novel method for integrating probabilistic shielding into offline RL within the SPI framework. We construct a shield from offline data and enforce it onto the baseline policy and during policy improvement, such that with high probability, actions that are likely to lead to unsafe outcomes are disabled. Thereby, we circumvent assumptions regarding (1) the safety of the baseline policy that is common in the SPI literature, and (2) the availability of a sufficient safety-relevant model, which is common in the shielding literature. Empirical results demonstrate that shielded SPI methods improve both average and worst-case performance across the four benchmark environments. Moreover, they consistently outperform their non-shielded counterparts when data is scarce. In future work,

we will bridge our computation of the shield from a dataset to the online RL setting.

ACKNOWLEDGMENTS

This work has been partially funded by the ERC Starting Grant DEUCE (101077178) and the ORLEANS project from the Interdisciplinary Research Platform of Radboud University.

REFERENCES

- [1] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. 2018. Safe Reinforcement Learning via Shielding. In *AAAI*. AAAI Press, 2669–2678.
- [2] Christel Baier and Joost-Pieter Katoen. 2008. *Principles of model checking*. MIT Press.
- [3] Richard Bellman. 1957. A Markovian Decision Process. *Indiana University Mathematics Journal* 6 (1957), 679–684. <https://api.semanticscholar.org/CorpusID:123329493>
- [4] Federico Bianchi, Edoardo Zorzi, Alberto Castellini, Thiago D. Simão, Matthijs T. J. Spaan, and Alessandro Farinelli. 2024. Scalable Safe Policy Improvement for Factored Multi-Agent MDPs. In *ICML*. OpenReview.net.
- [5] Asger Horn Brorholt, Andreas Holck Høeg-Petersen, Kim Guldstrand Larsen, and Christian Schilling. 2024. Efficient Shield Synthesis via State-Space Transformation. In *AIsoLA (Lecture Notes in Computer Science, Vol. 15217)*. Springer, 206–224.
- [6] Asger Horn Brorholt, Peter Gjørl Jensen, Kim Guldstrand Larsen, Florian Lorber, and Christian Schilling. 2023. Shielded Reinforcement Learning for Hybrid Systems. In *AIsoLA (Lecture Notes in Computer Science, Vol. 14380)*. Springer, 33–54.
- [7] Asger Horn Brorholt, Kim Guldstrand Larsen, and Christian Schilling. 2025. Compositional Shielding and Reinforcement Learning for Multi-Agent Systems. In *AAMAS*. International Foundation for Autonomous Agents and Multiagent Systems / ACM, 399–407.
- [8] Steven Carr, Nils Jansen, Sebastian Junges, and Ufuk Topcu. 2023. Safe Reinforcement Learning via Shielding under Partial Observability. In *AAAI*. AAAI Press, 14748–14756.
- [9] Alberto Castellini, Federico Bianchi, Edoardo Zorzi, Thiago D. Simão, Alessandro Farinelli, and Matthijs T. J. Spaan. 2023. Scalable Safe Policy Improvement via Monte Carlo Tree Search. In *ICML (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 3732–3756.
- [10] Yash Chandak, Scott M. Jordan, Georgios Theodorou, Martha White, and Philip S. Thomas. 2020. Towards Safe Policy Improvement for Non-Stationary MDPs. In *NeurIPS*.
- [11] Edwin Hamel-De le Court, Francesco Belardinelli, and Alexander W. Goodall. 2025. Probabilistic Shielding for Safe Reinforcement Learning. doi:10.48550/arXiv.2503.07671 arXiv:2503.07671 [stat].
- [12] Christian Dehnert, Sebastian Junges, Joost-Pieter Katoen, and Matthias Volk. 2017. A Storm is Coming: A Modern Probabilistic Model Checker. In *Computer Aided Verification (CAV) (Lecture Notes in Computer Science, Vol. 10427)*. Springer, 592–600.
- [13] Klaus Dräger, Vojtech Forejt, Marta Z. Kwiatkowska, David Parker, and Mateusz Ujma. 2014. Permissive Controller Synthesis for Probabilistic Systems. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (Lecture Notes in Computer Science, Vol. 8413)*. Springer, 531–546.
- [14] Kasper Engelen, Guillermo A. Pérez, and Marnix Suilen. 2025. Data-Efficient Safe Policy Improvement Using Parametric Structure. *CoRR* abs/2507.15532 (2025).
- [15] Damien Ernst, Pierre Geurts, and Louis Wehenkel. 2003. Iteratively Extending Time Horizon Reinforcement Learning. In *ECML (Lecture Notes in Computer Science, Vol. 2837)*. Springer, 96–107.
- [16] Mohammad Ghavamzadeh, Marek Petrik, and Yinlam Chow. 2016. Safe Policy Improvement by Minimizing Robust Baseline Regret. In *NIPS*. 2298–2306.
- [17] Alexander W. Goodall and Francesco Belardinelli. 2023. Approximate Model-Based Shielding for Safe Reinforcement Learning. In *ECAI (Frontiers in Artificial Intelligence and Applications, Vol. 372)*. IOS Press, 883–890.
- [18] Alexander Hans and Steffen Udluft. 2009. Efficient Uncertainty Propagation for Reinforcement Learning with Limited Data. In *ICANN (1) (Lecture Notes in Computer Science, Vol. 5768)*. Springer, 70–79.
- [19] Chloe He, Borja G. León, and Francesco Belardinelli. 2022. Do Androids Dream of Electric Fences? Safety-Aware Reinforcement Learning with Latent Shielding. In *SafeAI@AAAI (CEUR Workshop Proceedings, Vol. 3087)*. CEUR-WS.org.
- [20] Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30. <http://www.jstor.org/stable/2282952>
- [21] Garud N. Iyengar. 2005. Robust Dynamic Programming. *Math. Oper. Res.* 30, 2 (2005), 257–280.
- [22] Nils Jansen, Bettina Könighofer, Sebastian Junges, Alex Serban, and Roderick Bloem. 2020. Safe Reinforcement Learning Using Probabilistic Shields (Invited Paper). In *CONCUR (LIPIcs, Vol. 171)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 3:1–3:16.
- [23] Sebastian Junges, Nils Jansen, Christian Dehnert, Ufuk Topcu, and Joost-Pieter Katoen. 2016. Safety-Constrained Reinforcement Learning for MDPs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (Lecture Notes in Computer Science, Vol. 9636)*. Springer, 130–146.
- [24] Bettina Könighofer, Roderick Bloem, Nils Jansen, Sebastian Junges, and Ste fan Pranger. 2025. Shields for safe reinforcement learning. *CACM* (2025).
- [25] Marta Kwiatkowska, Gethin Norman, and David Parker. 2011. PRISM 4.0: Verification of Probabilistic Real-Time Systems. In *Computer Aided Verification (CAV) (Lecture Notes in Computer Science, Vol. 6806)*. Springer, 585–591.
- [26] Romain Laroche, Paul Trichelair, and Remi Tachet des Combes. 2019. Safe Policy Improvement with Baseline Bootstrapping. In *ICML (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 3652–3661.
- [27] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. 2020. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. *CoRR* abs/2005.01643 (2020).
- [28] Tobias Meggendorfer, Maximilian Weininger, and Patrick Wienhöft. 2025 (to appear, preprint at <https://arxiv.org/abs/2404.05424>). What Are the Odds? Improving the foundations of Statistical Model Checking. In *QEST + FORMATS*.
- [29] Daniel Melcer, Christopher Amato, and Stavros Tripakis. 2024. Shield Decentralization for Safe Reinforcement Learning in General Partially Observable Multi-Agent Environments. In *AAMAS*. International Foundation for Autonomous Agents and Multiagent Systems / ACM, 2384–2386.
- [30] Kimia Nadjahi, Romain Laroche, and Rémi Tachet des Combes. 2019. Safe Policy Improvement with Soft Baseline Bootstrapping. In *ECML/PKDD (3) (Lecture Notes in Computer Science, Vol. 11908)*. Springer, 53–68.
- [31] Arnab Nilim and Laurent El Ghaoi. 2005. Robust Control of Markov Decision Processes with Uncertain Transition Matrices. *Oper. Res.* 53, 5 (2005), 780–798.
- [32] Matteo Pirodda, Marcello Restelli, Alessio Pecorino, and Daniele Calandriello. 2013. Safe Policy Iteration. In *ICML (3) (JMLR Workshop and Conference Proceedings, Vol. 28)*. JMLR.org, 307–315.
- [33] Stefan Pranger, Bettina Könighofer, Martin Tappler, Martin Deixelberger, Nils Jansen, and Roderick Bloem. 2021. Adaptive Shielding under Uncertainty. In *ACC*. IEEE, 3467–3474.
- [34] Alberto Puggelli, Wenchao Li, Alberto L. Sangiovanni-Vincentelli, and Sanjit A. Seshia. 2013. Polynomial-Time Verification of PCTL Properties of MDPs with Convex Uncertainties. In *CAV (Lecture Notes in Computer Science, Vol. 8044)*. Springer, 527–542.
- [35] Martin L. Puterman. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley.
- [36] Harsh Satija, Philip S. Thomas, Joelle Pineau, and Romain Laroche. 2021. Multi-Objective SPIBB: Seldonian Offline Policy Improvement with Safety Constraints in Finite MDPs. In *NeurIPS*. 2004–2017.
- [37] Daniel Schneegass, Alexander Hans, and Steffen Udluft. 2010. *Uncertainty in Reinforcement Learning - Awareness, Quantisation, and Control*. doi:10.5772/10250
- [38] Philipp Scholl, Felix Dietrich, Clemens Otte, and Steffen Udluft. 2022. Safe Policy Improvement Approaches and Their Limitations. In *ICAART (Revised Selected Paper) (Lecture Notes in Computer Science, Vol. 13786)*. Springer, 74–98.
- [39] Thiago D. Simão, Romain Laroche, and Rémi Tachet des Combes. 2020. Safe Policy Improvement with an Estimated Baseline Policy. In *AAMAS*. International Foundation for Autonomous Agents and Multiagent Systems, 1269–1277.
- [40] Marnix Suilen, Thiago D. Simão, David Parker, and Nils Jansen. 2022. Robust Anytime Learning of Markov Decision Processes. In *NeurIPS*.
- [41] Martin Tappler, Stefan Pranger, Bettina Könighofer, Edi Muskard, Roderick Bloem, and Kim G. Larsen. 2022. Automata Learning Meets Shielding. In *ISO LA (1) (Lecture Notes in Computer Science, Vol. 13701)*. Springer, 335–359.
- [42] Philip S. Thomas, Georgios Theodorou, and Mohammad Ghavamzadeh. 2015. High-Confidence Off-Policy Evaluation. In *AAAI*. AAAI Press, 3000–3006.
- [43] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. 2024. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032 [cs.LG] <https://arxiv.org/abs/2407.17032>
- [44] Patrick Wienhöft, Marnix Suilen, Thiago D. Simão, Clemens Dubsaff, Christel Baier, and Nils Jansen. 2023. More for Less: Safe Policy Improvement with Stronger Performance Guarantees. In *IJCAI*. ijcai.org, 4406–4415.
- [45] Wolfram Wiesemann, Daniel Kuhn, and Berç Rustem. 2013. Robust Markov Decision Processes. *Math. Oper. Res.* 38, 1 (2013), 153–183.
- [46] Eric M. Wolff, Ufuk Topcu, and Richard M. Murray. 2012. Robust control of uncertain Markov Decision Processes with temporal logic specifications. In *CDC*. IEEE, 3372–3379.

Table 2: The dimensions of the benchmarks and full range of hyperparameters used in the experiments.

Benchmarks	$ S $	$ A $	N_Λ	ν	ϵ	θ	κ	η	α	ξ
<i>Random MDPs</i>	50	4	3	0.1	0.5	0.2	0.05	0.1	5	1×10^{-8}
<i>Wet Chicken</i>	26	5	7	0.05	0.05	0.2	0.5	0.1	2	1×10^{-8}
<i>Frozen Lake</i>	64	4	3	1	0.5	0.2	0.02	0.1	5	1×10^{-8}
<i>Pacman</i>	118k	4	3	1	0.5	0.01	0.01	0.1	10	1×10^{-8}

A ESTIMATING THE BASELINE POLICY

When using SPI algorithms, it is typically assumed that the baseline policy is given. However, Simão et al. [39] previously demonstrated that this assumption can be relaxed for SPIBB, allowing an estimated baseline to be used in its place. The estimation is done by using the occurrences of states $N_{\mathcal{D}}(s)$ and state-action pairs $N_{\mathcal{D}}(s, a)$ from the given dataset of trajectories $\mathcal{D}$. For each state, we normalize these counts to obtain a probability distribution over actions. If a state has not been visited in the dataset, we default to a uniform distribution over actions. The estimated baseline policy $\hat{\pi}_b$ is:

$$\hat{\pi}^b(a | s) = \begin{cases} N_{\mathcal{D}}(s, a) / N_{\mathcal{D}}(s) & \text{if } N_{\mathcal{D}}(s) > 0, \\ 1/|A| & \text{otherwise.} \end{cases} \quad (8)$$

In the paper, we denote the baseline policy by π^b as if it were known. Yet, we do not make this assumption in practice and instead use the estimated baseline $\hat{\pi}^b \approx \pi^b$ in our experiments.

B POINT ESTIMATES

PAC learning is a standard method for estimating transition functions of MDPs. The process begins by obtaining a point estimate $\hat{T}$, e.g., via maximum likelihood or otherwise. In our implementation, we use maximum a posteriori probability (MAP) estimates. Suppose we have the state-action pair (s, a) with m successor states, and each successor state s'_i is observed k_i times. Without loss of generality, for brevity we assume here that each (s, a) pair has m successor states. We use the Dirichlet distribution $\text{Dir}(T(\cdot | s, a) | \alpha_1, \dots, \alpha_m) \propto \prod_{i=1}^m T(s'_i | s, a)^{\alpha_i - 1}$ as a conjugate prior to the multinomial likelihood. Because it is a conjugate prior, we do not need to compute the posterior distribution explicitly. Instead, it is sufficient to update the parameters of the prior Dirichlet distribution. This way, we get the posterior distribution $\text{Dir}(T(\cdot | s, a) | \alpha_1 + k_1, \dots, \alpha_m + k_m)$, where k_i is the number of times successor state $i \in \{1, \dots, m\}$ has been observed. The MAP estimate can now be found by taking the mode:

$$\tilde{T}(s'_i | s, a) = \frac{\alpha_i + k_i - 1}{\sum_{j=1}^m (\alpha_j + k_j) - m} \quad (9)$$

This estimated transition function is likely to differ from the actual transition function. Yet the point estimate does not account for the uncertainty in the estimate. Therefore, as is common, we turn this point estimate into a confidence interval using Hoeffding’s inequality [20].

C COMPLETE PLOTS

For some of the plots shown in Section 5.1, the y-axis values have been clipped for better visualization:

- The Wet Chicken y-axis is clipped at -60,
- The Frozen Lake y-axis is clipped at -5, and
- The Random MDPs y-axis is clipped at -4.

We opted for clipping because the CVaR performance was very low in these cases, which would have otherwise made the plots difficult to read and interpret.

Included in this appendix are two figures that plot the complete data. Fig. 3 shows the average performance with error bars representing 95% confidence intervals. Fig. 4 shows the unclipped 1%-CVaR data.

D BENCHMARKS

We use four different environments to benchmark the performance of our approach. Each of these benchmarks, along with its baseline policy and safety specifications, will be explained in this subsection. The baseline policies are estimated from data, as explained in Section A.

Random MDPs. Our first experimental environment is a slightly modified version of the Random MDPs benchmark, as introduced in Nadjah et al. [30]. It consists of a randomly generated MDP with $|S| = 50$ states. Each state offers four actions, which can lead to four different states, with probabilities determined randomly. The agent starts in s_0 , and the goal state is selected to minimize the optimal value function. The environment also contains five randomly selected sink states, or “traps”. These trapped states are chosen such that all remain reachable

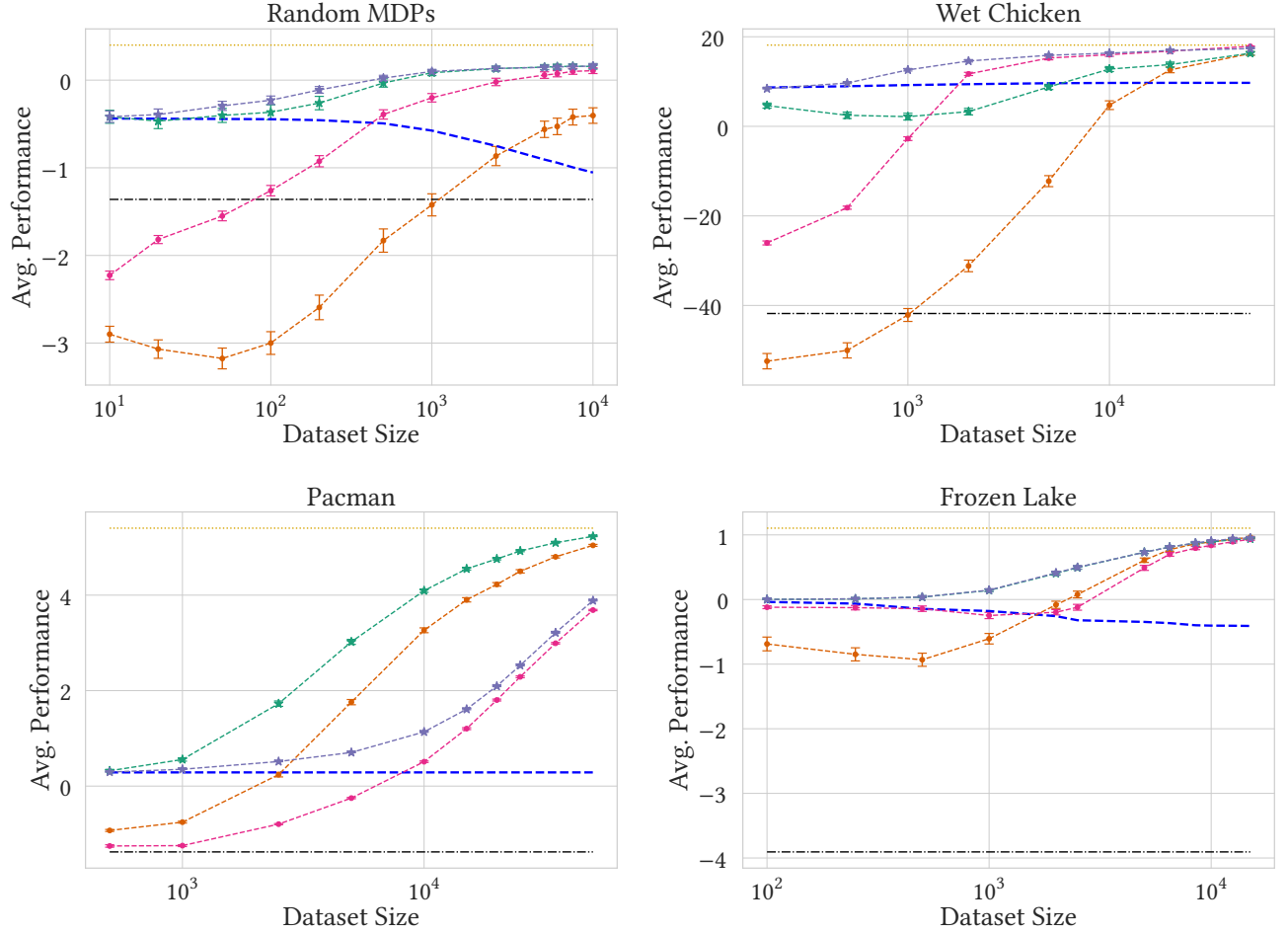


Figure 3: Full-range plots of the average performance of the shielded and non-shielded methods plotted against the number of trajectories in the dataset $\mathcal{D}$ (log-scale), including error bars representing 95% confidence intervals.

from the initial state. This modification was necessary as the original benchmark lacked unsafe states. All transitions yield a reward of 0 except for those leading into the final state or a trap state. A new random MDP is generated for each iteration.

The safety specification is defined as $\varphi = (\neg \text{trap} \cup \text{goal})$, requiring the agent to reach the goal while avoiding trap states. Here, the heuristic policy π^h used to construct the baseline policy is the optimal policy $\pi^* \in \Pi$.

Wet Chicken. The Wet Chicken benchmark [18, 38] models an agent controlling a boat on a river adjacent to a waterfall. We follow the description of Scholl et al. [38]. The objective is to stay as close to the edge of the waterfall without falling off. This river is represented by a 5×5 grid world. If the x position of the boat ever exceeds 5, it falls over the waterfall, its position resets to $(0, 0)$, and a large negative reward is obtained. Otherwise, the reward obtained in each subsequent timestep is equal to the x-position of the boat. The agent can perform a set of predefined actions: it can drift with no movement, corresponding to an action vector of $(a_x, a_y) = (0, 0)$, paddle back, representing a stronger motion away from the waterfall, with $(a_x, a_y) = (-2, 0)$, steer right, with $(a_x, a_y) = (0, 1)$, or steer left, with $(a_x, a_y) = (0, -1)$. The boat’s movement is also influenced by the stream $v_t = y_t \frac{3}{5}$ and turbulence $b_t = 3.5 - v_t$, where the turbulence direction $\tau_t \in [-1, 1]$ is sampled uniformly at random each timestep. The resulting dynamics are $(x_{t+1}, y_{t+1}) = \text{round}(x_t + a_x + v_t + \tau_t b_t), \text{round}(y_t + a_y)$. The safety specification is $\varphi = (\neg \text{waterfall} \cup \text{edge})$, requiring the agent to reach the states adjacent to the waterfall without falling.

Frozen Lake. We employ a modified version of the Frozen Lake environment from the Gymnasium library [43]. The task involves navigating an 8×8 grid to reach a goal state while avoiding hazardous hole (sink) states. Reaching such a state results in a negative reward, and reaching

the goal gives a positive reward. The agent can move in the four cardinal directions, but due to the slippery surface, there is a chance of deviating from the intended direction at a perpendicular angle.

The safety specification is defined as $\varphi = (\neg \text{hole} \cup \text{goal})$, requiring the agent to reach the goal without first falling into a hole.

The heuristic baseline policy π_H selects downward and rightward actions with equal probability. The final baseline policy π_b is formed by mixing π_H with the uniformly random policy $\tilde{\pi}$, weighted by parameter ϵ .

Simplified Pacman. We use an environment that models a simplified Pacman game, inspired by Alshiekh et al. [1]. The environment is a 7×7 grid maze where the agent starts in the bottom-left corner and aims to reach the top-right goal state. The agent can move in four cardinal directions: up, right, down, and left. Two ghost agents also move randomly throughout the maze. If the agent attempts to move into a wall, its position is reset to the initial state. If the agent collides with a wall, it is reset to the initial state. A collision with a ghost causes the agent to be eaten, resulting in a negative reward and terminating the episode. See Fig. 5 for a visual representation of the environment.

The state space size is $(7^2)^3 = 117649$, as the positions of the agent and two ghosts must be tracked. Using this environment, we demonstrate the scalability of our approach to large state spaces. The safety specification is $\varphi = (\neg \text{eaten} \cup \text{goal})$, requiring the agent to reach the goal while avoiding ghosts. The heuristic baseline policy π_H selects upward and rightward actions with equal probability, and disallows actions that would result in collisions with walls. The final baseline policy π_b is a mixture of π_H and the uniformly random policy $\tilde{\pi}$, weighted by parameter ϵ .

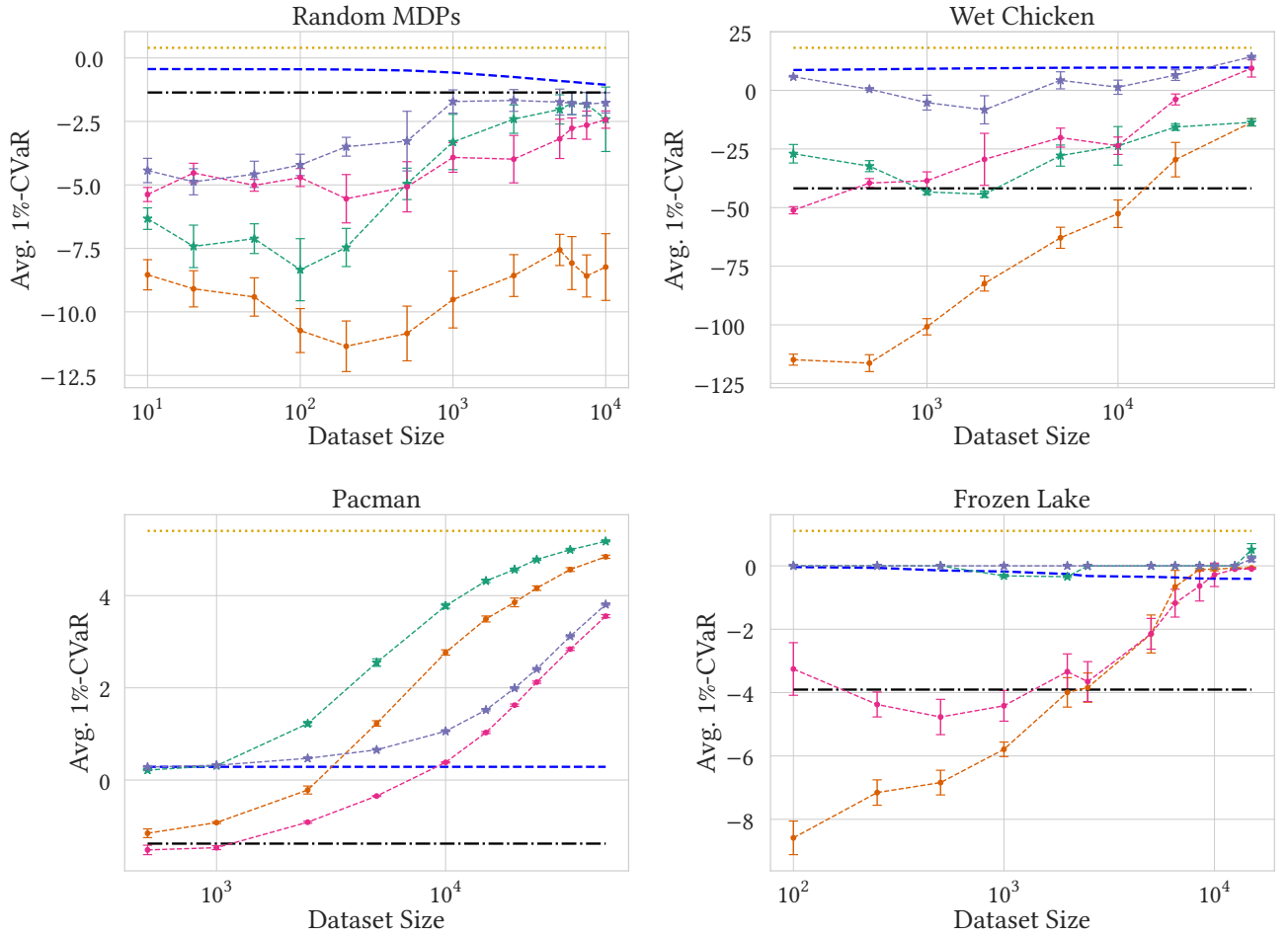


Figure 4: Full-range plots of the 1%-CVaR performance of the shielded and non-shielded methods plotted against the number of trajectories in the dataset $\mathcal{D}$ (log-scale). Unlike the clipped versions shown in the main text, these plots display the entire data range

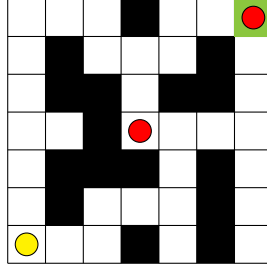


Figure 5: Starting position of the pacman environment

D.1 DUIPI

The *diagonal approximation of uncertainty incorporating policy iteration* (DUIPI) [37] algorithm penalizes the reward of state-action pairs with high uncertainty by calculating the variance.

In DUIPI, the variance of the action value $\text{Var}(Q_M^\pi(s, a))$ is computed by considering the variance of the estimated transition function $\hat{T}$. In the PE step, this variance penalizes the expected reward: $U(s, a) = Q_M^\pi(s, a) - \nu \sqrt{\text{Var}(Q_M^\pi(s, a))}$, where ν is a hyperparameter that determines the degree to which the variance influences the uncertainty. We provide details on the computation of U and $\hat{T}$ in Section D.1. The resulting probabilistic guarantee is that with probability $1 - F(\zeta)$, the true action-value Q_M^π exceeds the value U :

$$\Pr(Q_M^\pi(s, a) > U(s, a)) = 1 - F(\zeta), \quad (10)$$

where $F(\zeta) = \Pr(Z \leq \zeta)$ is the cumulative density function for a standard normal distributed Z . Here, it is assumed that the action-value function is normally distributed, and that covariance between state-action pairs is negligible. We refer to Schneegass et al. [37] and Scholl et al. [38] for a more extensive discussion.

During policy iteration, DUIPI updates the baseline policy by iteratively increasing the probability of the action with the highest penalized action-value U . Actions not selected as best are adjusted accordingly to maintain a valid probability distribution. The rate of change decreases as the iteration count increases. Formally, at policy iteration step $t \in \mathbb{N}$, we update the policy π' , using $a_U^*(s) = \arg\max_{a \in A} U(s, a)$, as:

$$\pi'(a | s) = \begin{cases} \min(\pi(a | s) + 1/t, 1) & \text{if } a \in a_U^*(s), \\ \frac{\max(1 - \pi(a_U^*(s) | s) - 1/t, 0)}{1 - \pi(a_U^*(s) | s)} & \text{otherwise.} \end{cases} \quad (11)$$

As it proved more performant, we opted for the Bayesian version of DUIPI, estimating $\hat{T}$ with Dirichlets, and, instead of randomly initializing the initial policy used in the policy iteration procedure, we used the baseline policy π^b as an initial guess.

Shielded-DUIPI. For shielded-DUIPI, we amend the action-value function by assigning the minimum reward to any unsafe state-action pair. Recall U from Section D.1. Then, this results in the shielded value $Q_{\square}$, with:

$$U_{\square}(s, a) = \begin{cases} U(s, a) & \text{if } (s, a) \in SA_{\square}, \\ -\infty & \text{otherwise.} \end{cases} \quad (12)$$

Thus, unsafe actions cannot be selected. In this case, we use the shielded baseline policy $\pi_{\square}^b$ as the initial guess instead of the original baseline policy π^b .

Rather than using a full covariance-based formulation, which is computationally expensive for large state spaces, DUIPI employs an approximate algorithm that utilizes only the diagonal of the covariance matrix. We assume independence between components. Schneegass et al. [37] found that this approximation can be made for larger MDPs with more than 18 states and 4 actions, which is the case for all environments used in this paper. When neglecting correlations, we can estimate the variance of a transformation f on vector X using [37, 38]:

$$\text{Var}(f(X)) \approx \sum_i \left(\frac{\partial f}{\partial X_i} \right)^2 \text{Var}(X_i) \quad (13)$$

When estimating the uncertainty of the action-value function Q_M^π , we must account for the variance introduced by T and R , as well as the variance propagated through V_M^π . We can therefore estimate the variance of Q_M^π using [37, 38]:

$$\begin{aligned}
\text{Var}(Q_{\hat{M}}^{\pi}(s, a)) &\approx \sum_{s'} \gamma^2 \hat{T}(s' | s, a)^2 \text{Var}(V_{\hat{M}}^{\pi}(s')) \\
&+ \sum_{s'} \left(\hat{R}(s, a, s') + \gamma V_{\hat{M}}^{\pi}(s') \right)^2 \text{Var}(\hat{T}(s' | s, a)) \\
&+ \sum_{s'} \hat{T}(s' | s, a)^2 \text{Var}(\hat{R}(s, a, s'))
\end{aligned} \tag{14}$$

Each term corresponds to a distinct source of uncertainty.